



INSTITUTO POLITÉCNICO NACIONAL
Escuela Superior de Cómputo



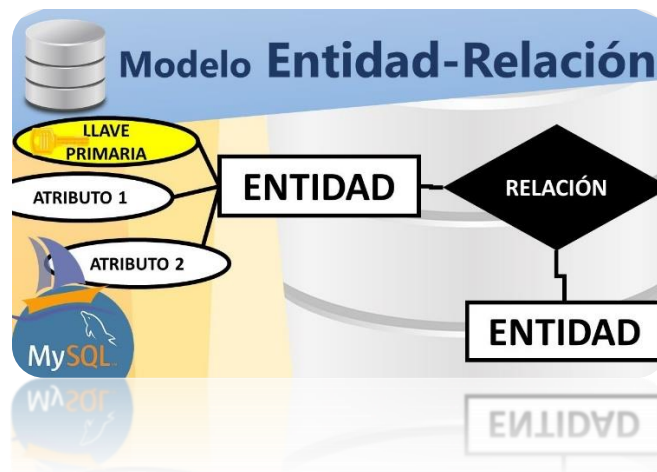
Investigación correspondiente a los ejercicios 1 y 2 (parte teórica) de la práctica 1 “Modelo Entidad Relación”

Profesor: Gabriel Hurtado Avilés

Alumno: Marco Dichi

Materia: Bases de datos

Grupo: 3CV2



Ejercicio 1. Control de versiones con Git y GitHub (Parte A. Investigación)

1. ¿Qué es un sistema de control de versiones? Y qué problema concreto resuelve en un trabajo en equipo.

Un sistema de control de versiones (VCS, Version Control System) es una herramienta que permite registrar y administrar los cambios realizados en los archivos de un proyecto a lo largo del tiempo. Su función no se limita a guardar diferentes copias de un archivo. Permite conocer qué cambió, cuándo cambió, quién realizó el cambio y, dependiendo del sistema, recuperar versiones anteriores.

Esta herramienta es muy útil a la hora de trabajar proyectos o tareas en equipo. Para poder dimensionar la forma en la que un sistema control de versiones ayuda en trabajos colaborativos, podemos imaginar el siguiente escenario:

Imagine que 4 personas trabajan sobre un proyecto.

Sin control de versiones podrían ocurrir situaciones como:

- proyecto_final.docx
- proyecto_final2.docx
- proyecto_final_ahora_si.docx
- proyecto_final_definitivo.docx
- proyecto_final_definitivo_2.docx

En cambio, con el sistema control de versiones podemos ver como ya se mencionó anteriormente, qué fue lo que cambió, cuándo y quién realizó dicho cambio.

Siguiendo con el ejemplo, podríamos ver que el primer documento es el primero que se generó, sobre los posteriores podríamos ver qué es lo que cambia respecto a los anteriores y quiénes fueron los integrantes que realizaron cada cambio dentro de cada versión del documento.

2. Git vs GitHub, cuál es la diferencia

Git

Git es un sistema de control de versiones distribuido.

Es un software que se instala y ejecuta en tu computadora y permite controlar las versiones de un proyecto.

Tu computadora



Git



Repositorio local

Puedes crear commits, ramas, fusionarlas, consultar el historial, etc., sin necesidad de estar conectado a Internet.

GitHub

GitHub es una plataforma en línea que permite alojar repositorios Git y colaborar sobre ellos.

Tu computadora



Git



Repositorio local



GitHub



Repositorio remoto

Git y GitHub, por lo tanto, cumplen funciones diferentes:

Git	GitHub
Software de control de versiones	Plataforma de alojamiento y colaboración
Funciona de manera local	Funciona principalmente en línea
Administra historial y cambios	Facilita compartir y colaborar
Permite commits y branches	Permite Pull Requests, revisión, issues, etc.

En resumen:

Git controla las versiones; GitHub permite alojar y colaborar sobre repositorios Git.

3. Defina, con sus palabras y con un ejemplo: repositorio, confirmación (commit), rama (branch), fusión (merge), conflicto de fusión, Pull Request, archivo .gitignore y archivo README.

Repositorio

Un repositorio es el espacio donde se almacena un proyecto y su historial de versiones.

Puede ser local o remoto.

Ejemplo: El repositorio de mi practica1 contendrá los documentos, archivos de configuración, evidencias y demás elementos de la práctica.

Commit (Confirmación)

Un commit es un registro de cambios que se guarda en el historial de Git.

Un commit representa un punto concreto en la evolución del proyecto.

Por ejemplo:

```
Commit 1 → Se crea README
Commit 2 → Se agrega .gitignore
Commit 3 → Se agrega documentación
```

Cada commit debe tener un mensaje que describa razonablemente qué se modificó.

Branch (Rama)

Una rama es una línea independiente de desarrollo dentro de un repositorio.

Permite realizar cambios sin modificar directamente la rama principal.

Por ejemplo:

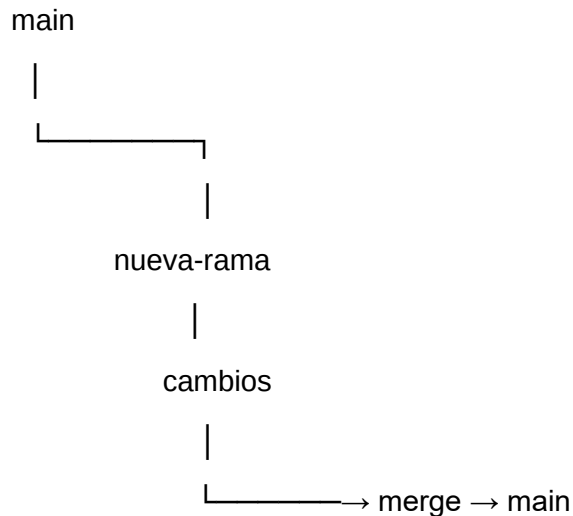
```
main
|
|— commit
|
|— branch: mejorar-readme
    |
    |— cambio
    |— commit
```

Esto resulta especialmente útil cuando varias personas trabajan simultáneamente o cuando se quiere probar una modificación antes de incorporarla al proyecto principal.

Merge (Fusión)

Un merge consiste en integrar los cambios realizados en una rama dentro de otra.

Por ejemplo:



En la práctica, precisamente se tendrá que crear una rama, modificar algo y posteriormente fusionarla.

Merge Conflict (Conflicto de fusión)

Un conflicto de fusión ocurre cuando Git no puede determinar de forma automática cómo combinar cambios realizados en una misma parte de un archivo.

Por ejemplo, imaginemos que dos ramas modifican la misma línea:

Rama A:

nombre = "Marco"

Rama B:

nombre = "Gabriel"

En estos casos Git no puede decidir automáticamente cuál debe conservar.

Entonces requiere que el usuario revise el conflicto y determine cuál modificación debe mantenerse.

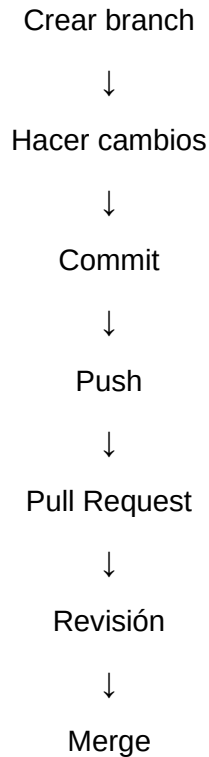
Pull Request (solicitud de extracción)

Un Pull Request (PR) es una solicitud para incorporar los cambios de una rama a otra, normalmente hacia la rama principal.

No es simplemente "hacer merge".

El Pull Request permite que los cambios sean revisados antes de fusionarse.

Un flujo típico sería:



En la parte práctica se tendrá que realizar exactamente este proceso.

[.gitignore file \(archivo gitignore\)](#)

.gitignore es un archivo que especifica qué archivos o carpetas Git debe ignorar y, por lo tanto, no incluir normalmente en los commits.

Por ejemplo, un proyecto puede generar archivos temporales o configuraciones locales que no deberían formar parte del repositorio.

Un .gitignore adecuado evita subir accidentalmente este tipo de archivos.

[README file \(archivo README\)](#)

Un archivo README, normalmente README.md, es un documento que proporciona información sobre el proyecto.

Puede explicar:

- Qué es el proyecto
- Quién lo realizó
- Cómo utilizarlo
- Su estructura
- Instrucciones
- Información relevante

En la práctica, el profesor pide específicamente que el README.md incluya:

- Nombre completo

- Grupo
- Carrera
- Índice de la práctica

4. En qué consiste un flujo de trabajo basado en ramas y por qué el código se revisa entre pares antes de fusionarse.

Un flujo basado en ramas consiste en utilizar diferentes ramas para desarrollar cambios de manera separada antes de integrarlos a la rama principal.

Por ejemplo:

Cada cambio puede desarrollarse de manera independiente. Una vez terminado, se puede solicitar su integración mediante un Pull Request.

¿Por qué revisar entre pares?

La revisión entre pares permite que otra persona examine los cambios antes de incorporarlos al proyecto principal.

Esto ayuda a:

- Detectar errores
- Encontrar problemas de diseño
- Comprobar que se cumplan los requisitos
- Mejorar la calidad del código
- Reducir la posibilidad de introducir errores en la rama principal

Por eso el Pull Request no es únicamente un mecanismo para "pedir permiso para hacer merge": también funciona como mecanismo de revisión y colaboración.

Ejercicio 2. El sistema gestor en un contenedor: Docker (Parte A. Investigación)

1. Qué es un contenedor y en qué se diferencia de una máquina virtual, en términos de arranque, tamaño y aislamiento.

Un contenedor es un entorno aislado que permite ejecutar una aplicación junto con los elementos necesarios para funcionar, utilizando el kernel del sistema operativo anfitrión.

Una máquina virtual (VM), en cambio, virtualiza una computadora completa y ejecuta su propio sistema operativo invitado.

Comparativa:

Aspecto	Contenedor	Máquina Virtual
Arranque	Muy rápido (segundos), porque comparte el kernel del sistema operativo.	Lento (minutos), porque arranca un sistema operativo completo dentro de la VM.
Tamaño	Ligero (MBs), solo incluye la aplicación y dependencias.	Pesado (GBs), incluye todo un sistema operativo invitado.
Aislamiento	A nivel de procesos, comparten kernel pero están aislados en espacio de usuario.	A nivel de hardware virtualizado, cada VM tiene su propio SO completo.

2. Defina: imagen, contenedor, volumen y puerto publicado.

Imagen: Plantilla inmutable que contiene el sistema de archivos y dependencias necesarias para ejecutar una aplicación.

Contenedor: Un contenedor es una instancia ejecutable creada a partir de una imagen, es el proceso vivo que corre una aplicación.

La imagen sería el "molde" y el contenedor sería una instancia creada a partir de ese molde.

Volumen: Área de almacenamiento persistente que se monta en el contenedor para guardar datos más allá de su ciclo de vida.

Puerto publicado: Mapeo entre un puerto del host y un puerto del contenedor, para que la aplicación sea accesible desde fuera.

Ej: -p 5432:5432

El primer 5432 corresponde al puerto del host y el segundo al puerto dentro del contenedor.

Así, una aplicación como pgAdmin puede conectarse al PostgreSQL que está ejecutándose dentro del contenedor mediante el puerto publicado.

3. Por qué el volumen es indispensable y qué ocurre exactamente si no lo declara.

En el contexto de una base de datos, el volumen es necesario para que los datos no dependan exclusivamente del ciclo de vida del contenedor. Un contenedor puede eliminarse y posteriormente crearse otro a partir de la misma imagen. La imagen contiene PostgreSQL, pero no contiene los datos de una base de datos creada posteriormente.